

Problem 7 – Smart Birthday Reminder 🎂

Difficulty: ★★ Beginner

Overview

A family wants a simple application that helps them remember the birthdays of friends and relatives. Instead of checking a paper calendar, they want a smart assistant that stores birthdays, searches for people, calculates ages, and reminds them of upcoming birthdays.

Your task is to build a Smart Birthday Reminder using Streamlit.

Project Requirements

1. Store Birthday Records

Store all birthday records in an appropriate data structure.

Each record must contain:

- Person's Name
- Birthday
- Relationship

Available relationships:

Relationship

Family

Friend

Classmate

Coworker

Note: Choose the most appropriate Python data structure to store the birthday records.

2. Build the User Interface

Use Streamlit to create the application.

Required Text Functions

Use all of the following:

- `st.title()`
- `st.header()`
- `st.subheader()`
- `st.write()`
- `st.divider()`

3. Add a Birthday

Create a form that allows the user to enter:

- Person's Name
- Birthday
- Relationship

Use the following widgets:

Widget	Purpose
<code>st.text_input()</code>	Person's Name
<code>st.date_input()</code>	Birthday
<code>st.selectbox()</code>	Relationship
<code>st.button()</code>	Save Birthday

4. Validate User Input

Before saving the record, ensure that:

- The name is not empty.
- A birthday has been selected.
- A valid relationship has been chosen.

Display an appropriate message if the input is invalid.

5. Save Birthday Records

When the Save Birthday button is clicked:

- Validate the input.
 - Store the birthday record.
 - Display a success message.
-

6. Display All Birthdays

Create a page that displays every birthday record.

For each person, display:

- Name
- Birthday
- Current Age
- Relationship

Example

Ahmed

Birthday : 14/05/2010

Age : 15 Years

Relationship : Friend

7. Search for a Person

Create a search section.

Allow the user to search using a person's name.

If the person exists, display:

- Name
- Birthday
- Age
- Relationship

Otherwise, display an appropriate message.

The search should ignore uppercase and lowercase differences.

Bonus Mission ★

For every displayed birthday, calculate and display:

Days Remaining Until the Next Birthday

Example

Next Birthday In

42 Days

Required Status Elements

Use at least four of the following status elements.

Streamlit Function	Suggested Usage
<code>st.info()</code>	Display today's date or a birthday reminder tip in the sidebar.
<code>st.success()</code>	Birthday record saved successfully.
<code>st.warning()</code>	Notify the user when the selected birthday is today ("🎉 Today is Ahmed's birthday!").
<code>st.error()</code>	Display invalid input messages or "Person not found" after searching.

These usages naturally match each status element and avoid forcing students to invent situations where they don't fit.

Required Layout

Sidebar

Display:

- Today's Date
- Relationship List
- Birthday Tips

Tabs (Create three tabs).

Tab 1 - Add Birthday

Contains the birthday entry form.

Tab 2 - Birthday List

Display all birthday records.

Use `st.columns()` to organize the information in a clear and readable layout.

- Name
 - Birthday
 - Age
 - Relationship
 - Days Until Next Birthday (Bonus)
-

Tab 3 - Search

Contains the search box and displays the matching person's information.

- Name
 - Birthday
 - Age
 - Relationship
 - Days Until Next Birthday (Bonus)
-

Challenge Mission ★★

Create a Birthday Statistics page or section that displays:

- Total Number of People
 - Youngest Person
 - Oldest Person
 - Number of Family Members
 - Number of Friends
 - Number of Classmates
 - Number of Coworkers
-

Useful Hints

- Choose an appropriate data structure to store birthday records.
 - Create reusable functions such as:
 - `save_birthday()`
 - `display_birthdays()`
 - `search_person()`
 - `calculate_age()`
 - Use the `datetime` module to calculate ages.
 - Use `.lower()` when comparing names during a search.
 - Use `st.session_state` to preserve birthday records while the application is running.
-

Expected GUI

Smart Birthday Reminder

Sidebar

Today's Date

Relationship List

Birthday Tips

Tabs

[Add Birthday] [Birthday List] [Search]

[Add Birthday]

Name [Ahmed]

Birthday [14/05/2010]

Relationship [Friend ▼]

[Save Birthday]

 **Birthday saved successfully!**

[Birthday List]

Name : Ahmed

Age : 15 Years

Relationship : Friend

Next Birthday: 42 Days

[Search] [Enter Name]

Name : Ahmed

Age : 15 Years

Relationship : Friend

Next Birthday: 42 Days

Useful Modules

- datetime
-

Python Topics Covered

- Variables
 - Strings
 - Lists and/or Dictionaries
 - Conditional Statements
 - User-defined Functions
 - datetime
 - Date Calculations
 - String Methods (.lower())
 - st.session_state
-

Streamlit Topics Covered

Display

- `st.title()`
- `st.header()`
- `st.subheader()`
- `st.write()`
- `st.divider()`

Input Widgets

- `st.text_input()`
- `st.date_input()`
- `st.selectbox()`
- `st.button()`

Status Elements

- `st.info()`
- `st.success()`
- `st.warning()`
- `st.error()`

Layout

- `st.sidebar`
- `st.tabs()`
- `st.columns()`

Learning Objectives

After completing this project, you should be able to:

- Store structured information using an appropriate Python data structure.
- Work with dates using the datetime module.
- Calculate ages and perform basic date arithmetic.
- Search records using case-insensitive string comparison.
- Organize an application into reusable functions.
- Build a multi-tab Streamlit application that combines data entry, searching, and data presentation in a user-friendly interface.